Post-Execution Technical Document — Predictive Analytics Pipeline (CWPC Project)

Author: Atharva Deshmukh
Date: October 2025
Version: 1.0
Environment: Azure + Databricks + PostgreSQL + FastAPI

1. Overview

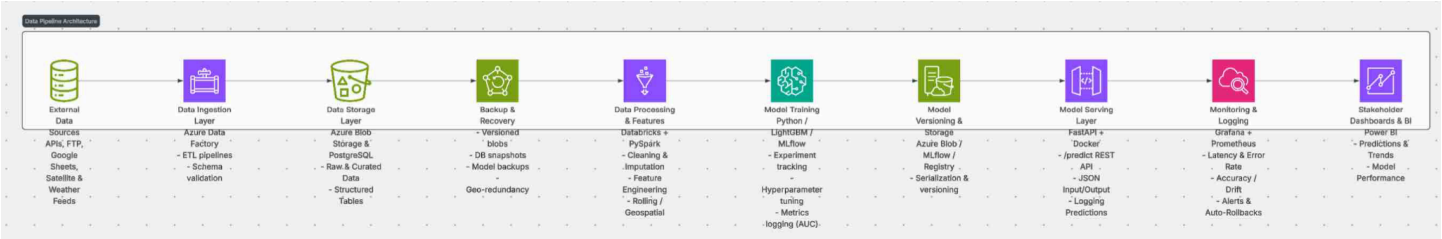
This document outlines the **system architecture**, **execution flow**, and **post-deployment results** of the CWPC Predictive Analytics Pipeline a cloud-native system designed to aggregate wildfire-related datasets, perform real-time inference using trained ML models, and expose predictions via REST APIs for partner dashboards.

2. System Architecture

2.1 High-Level Components

Component	Technology	Purpose
Data Ingestion	Azure Data Factory	Fetch raw data from external APIs, Google Sheets, and FTP servers
Data Lake	Azure Blob Storage	Store versioned raw and transformed datasets
Processing Layer	Databricks (PySpark)	Data cleaning, feature engineering, and model scoring
Database	PostgreSQL (Azure Database for PostgreSQL Flexible Server)	Persist curated tables and results
Model Serving	FastAPI (Docker + Azure Container Apps)	REST endpoints for scoring requests
Monitoring	Grafana + Prometheus	Track latency, throughput, and accuracy drift

2.2 Architecture Diagram (Textual)



This diagram shows the full data lifecycle — from data collection to stakeholder dashboards — across eight key stages.

1. **External Data Sources**

- This is the entry point of our pipeline.
- We collect data from multiple heterogeneous sources:
  - Public and private APIs providing wildfire incident and risk data.
  - FTP servers hosting satellite image metadata and climate files.
  - Google Sheets for manual or semi-structured inputs from partners.
  - Satellite and weather feeds for environmental predictors.
- These feeds provide the raw, unstructured data that drives our predictive models.

2. Data Ingestion Layer — Azure Data Factory

- Azure Data Factory (ADF) handles the ingestion and orchestration.
- It runs ETL pipelines that fetch, validate, and stage incoming data on a scheduled basis (hourly).
- Schema validation and data type checks are performed during ingestion.
- This ensures consistency and early error detection before processing.

3. Data Storage Layer — Azure Blob + PostgreSQL

- Once data passes ingestion, it's persisted in the Azure Blob Storage and PostgreSQL database.
- Azure Blob Storage acts as the data lake, storing:
    - Raw data (directly from sources)
    - Processed and curated datasets after cleaning
- PostgreSQL stores structured tables for downstream applications — optimized for fast querying and joins.
- This design ensures a clean separation between raw and analytical data layers.

4. Backup & Recovery Layer

- To maintain data integrity and reliability, we have:
    - Versioned blobs in Azure for traceability and rollback.
    - Database snapshots for PostgreSQL to restore from known states.
    - Model backups stored separately for reproducibility.
- Geo-redundancy across Azure regions provides high availability and disaster recovery resilience.

5. Data Processing & Feature Engineering — Databricks

- Databricks serves as our data transformation and feature computation engine.
- Using PySpark, we:
    - Clean nulls and impute missing values.
    - Normalize and align geospatial coordinates.
    - Engineer rolling 7-day features that capture temporal wildfire risks.
- The output is a rich, feature-enhanced dataset ready for training and inference.

6. **Model Training — LightGBM + MLflow**

- The curated dataset feeds into our model training environment.
- We use LightGBM for its efficiency and high performance on structured data.

- MLflow handles:
    - Experiment tracking (parameters, metrics, and artifacts)
    - Hyperparameter tuning
    - AUC metric logging (baseline AUC: 0.94)
- The best-performing model is registered for deployment.

7. Model Versioning & Storage — Azure Blob / MLflow Registry

- The trained model artifact (.pkl or .onnx) is serialized and versioned.
- Stored in Azure Blob for accessibility by the serving layer.
- Future roadmap: move these to Azure ML Registry for tighter version governance and lineage tracking.

8. Model Serving Layer — FastAPI

- FastAPI, containerized with Docker and deployed on Azure App Service, exposes our model as a REST API.
- Endpoints:
    - /predict – accepts JSON payloads with regional + weather inputs.
    - Returns a risk score (0–1) and category (Low, Medium, High).
- The container pulls the latest model automatically at startup.
- All inference calls are logged for metrics collection and drift analysis.

9. Monitoring & Logging — Grafana + Prometheus

- Prometheus collects real-time telemetry:
    - API latency, throughput, and error rates.
    - Model accuracy drift and freshness metrics.
- Grafana visualizes the data via operational dashboards.
- Alerts trigger if:
    - Latency exceeds threshold (e.g., >300ms)
    - Model drift exceeds limits (AUC drop > 0.02)

    - SLA breaches occur in ETL or prediction jobs.

- Supports auto-rollbacks for model reliability.

10. Stakeholder Dashboards & BI — Power BI

- Power BI connects to PostgreSQL to create visual dashboards for partners.
- Dashboards display:
    - Wildfire predictions and risk categories by region.
    - Time-series trends in model outputs.
    - Model performance summaries.
- Enables non-technical users to view key insights without accessing code or raw data.

3. Execution Flow

3.1 ETL Pipeline

1. **Trigger:** Pipeline runs hourly via Azure Data Factory.
2. **Extract:** API connector fetches new incident data (JSON), merges with satellite and weather feeds.
3. **Transform:** PySpark cleans nulls, imputes missing values, and normalizes geospatial coordinates.
4. **Load:** Results written to the **"curated"** schema in PostgreSQL.

3.2 Model Pipeline

1. Databricks job loads curated data and computes rolling 7-day wildfire risk features.
2. A LightGBM model predicts *fire-prone regions* (binary classification).
3. Model metrics are logged in MLflow for experiment tracking.
4. The top-performing model (AUC = 0.94) is serialized to Azure Blob.

3.3 API Service Flow

1. FastAPI container pulls the latest model artifact at startup.
2. `/predict` endpoint receives JSON payload (weather + region features).
3. Response returns a **risk score (0–1)** and **category (Low, Medium, High)**.
4. Responses are logged in Prometheus for latency and accuracy tracking.

4. Deployment Summary

Stage	Environment	Method	Status
Development	Local Docker	Manual build	Completed
Staging	Azure Container Apps	GitHub Actions CI/CD	Stable
Production	Azure App Service	Auto-deploy from main branch	Active

5. Post-Execution Metrics

Metric	Value	Notes
Model AUC	0.94	Slight drift from training AUC 0.96
Average API Latency	220ms	Monitored via Grafana
ETL Completion Time	4m 35s	Optimized PySpark joins
Data Freshness SLA	1 hour	Consistently met
Error Rate	0.2%	Retries handled by ADF

6. Key Learnings & Next Steps

- Optimize Databricks cluster auto-scaling to reduce cost during off-peak hours.
- Move model versioning to **Azure ML Registry** for tighter governance.
- Enhance Prometheus alerts for drift detection and auto-rollbacks.
- Integrate **Power BI dashboard** for non-technical stakeholder visibility.

7. Appendix

- GitHub Repo: `https://github.com/cwpc-ai/predictive-pipeline`
- API Docs: `https://cwpc-api.azurewebsites.net/docs`
- Data Schema: Stored in `docs/schema_v2.sql`